

Correção do exame de recurso POO 24/25

```
1.1- public class Conselheiro extends Juiz {  
    public Conselheiro(int c, String r, int a) {  
        super(c,r,a);  
    }  
    public double rendimento() {  
        double valor = 0.0;  
        for(Processo p : super.processos())  
            if(p.elementos().size() > 10)  
                valor += this.duracao(p) * 15.0;  
            else  
                valor += this.duracao(p) * 10.0;  
        return valor;  
    }  
    public int duracao(Processo p) {  
        return p.elementos().size() * 2;  
    }  
    public Conselheiro clone() {  
        Conselheiro tmp = new  
        Conselheiro(super.getCod(),super.getRes(),super.getAnos());  
        for(Processo p : super.processos())  
            tmp.add(p);  
        return tmp;  
    }  
}
```

1.2 – import java.util.List;

import java.util.ArrayList;

```

public class Proc_Exemplo implements Processo {

    private int cod, dias;

    private String local;

    private List<Integer> funcionarios;


    public Proc_Exemplo(int cod, String l, int d) {

        this.cod = cod;

        this.local = l;

        this.dias = d;

        this.funcionarios = new ArrayList<Integer>();

    }

    public int getCod() {

        return this.cod;

    }

    public String getLocal() {

        return this.local;

    }

    public List<Integer> elementos() {

        ArrayList<Integer> tmp = new ArrayList<Integer>();

        for(int x : this.funcionarios)

            tmp.add(x);

        return tmp;

    }

}

// por exemplo!

public class Juiz2 extends Juiz {

    public Juiz2(int c, String r, int a) {

        super(c,r,a);

    }

}

```

```
}
```

```
public double rendimento() {  
    return 100.00; // processo com valor fixo  
}
```

```
public int duracao(Processo p) {  
    return p.elementos().size(); // para ser diferente  
}
```

```
public Juiz2 clone() {  
    Juiz2 tmp = new Juiz2(super.getCod(),super.getRes(),super.getAnos());  
    for(Processo p : super.processos())  
        tmp.add(p);  
    return tmp;  
}  
}
```

2.1 - import java.util.*;

```
public class Tribunal {  
    private Map<Integer,Juiz> lista;  
    public Tribunal() {  
        this.lista = new HashMap<Integer,Juiz>();  
    }  
    public boolean add(Juiz j) {  
        if(j != null) {  
            if(this.lista.containsKey(j.getCod()))  
                return false;  
        }  
    }  
}
```

```

        else {
            this.lista.put(j.getCod(),j.clone());
            return true;
        }
    }
    else
        return false;
}

```

2.2. –

```

public String maior_residencia() {
    String max="";
    double max2=-1.0;
    for(Juiz j : this.lista.values())
        if(j.rendimento() > max2) {
            max = j.getRes();
            max2 = j.rendimento();
        }
    return max;
}

```

2.3 -

```

public Map<Integer,ArrayList<Integer>> funcionarioProcs() {
    HashMap<Integer,ArrayList<Integer>> tmp = new
    HashMap<Integer,ArrayList<Integer>>();
    for(Juiz j : this.lista.values()) {
        for(Processo p : j.processos())
            for(Integer f : p.elementos()) {

```

```

        ArrayList<Integer> novo = null;

        if(tmp.containsKey(f))

            novo = tmp.get(f);

        else {

            novo = new ArrayList<Integer>();

            tmp.put(f,novo);

        }

        novo.add(p.getCod());

    }

}

return tmp;

}

```

2.4 -

```

public int func_max() {

    HashMap<Integer,Integer> tmp = new HashMap<Integer,Integer>();

    for(Juiz j : this.lista.values()) {

        for(Processo p : j.processos()) {

            int dura = j.duracao(p);

            for(Integer f : p.elementos()) {

                int tot = 0;

                if(tmp.containsKey(f))

                    tmp.put(f, tmp.get(f) + dura);

                else {

                    tmp.put(f, dura);

                }

            }

        }

    }

}

```

```

    }

    // procurar maior valor de duracao

    int max = 0;

    int imax = -1;

    for(int f : tmp.keySet())

        if(tmp.get(f) > max) {

            max = tmp.get(f); imax = f;

        }

    return imax;

}

```

2.5.

```

public Set<Processo> ordemProcs(int cod) {

    TreeSet<Processo> conj = null;

    Juiz j = this.lista.get(cod);

    if(j != null) {

        Comparator<Processo> ord = (p1,p2) -> {

            // caso de empate, desempate por local lexicograficamente

            if(j.duracao(p1) == j.duracao(p2)) {

                return p1.elementos().size() - p2.elementos().size();

            }

            if(j.duracao(p1) < j.duracao(p2))

                return -1;

            else

                return 1;

        };

        conj = new TreeSet<Processo>(ord);
    }
}

```

```
        for(Processo p : j.processos())
            conj.add(p);
    }
    return conj;
}
```

2.6 -

* Mencionar pelo menos o seguinte:

* 1) get() implica execução de uma inserção numa tabela de hash.

* 2) em Java a classe HashMap implementa hashing via closed addressing i.e., chaining.

* 3) Explicar hashing e o uso da função de hash durante o get().

* 4) Explicar chaining na resolução das colisões com exemplos.

* 5) Usar exemplos.

* 6) Justificar porque esta estrutura de dados é uma boa ideia.

*

*/

}